CHAPTER IV

RESULTS AND DISCUSSIONS

This chapter presents the documented implementation of eSIMS: Smart School Incident Management System Using Rule-Based Algorithms for Incident Classification and Risk Assessment. The discussion follows the incident-management process from reporting and initial analysis to investigation, intervention, parent communication, and administrative reporting. It also describes the supporting functions for student records, user access, school settings, and activity monitoring.

The implementation findings are based on a read-only examination of project requirements, data models, services, and view models. The numerical evaluation in this chapter is an illustrative simulation prepared at the researcher's request; no respondent data were collected or supplied, and no formal runtime tests were completed for this documentation. All simulated values must be replaced with actual evidence before they are presented as research findings. The figures are source-derived workflow diagrams, not screenshots.

System Implementation Overview

eSIMS is a Windows desktop application developed in C# using Windows Presentation Foundation and the .NET 8 target framework. It stores its core records in a local SQLite database through Entity Framework Core. Its Model–View–ViewModel organization separates stored information, business operations, and interface state. The project also uses LiveCharts for visual reporting and QuestPDF for document generation. These components support an incident-management workstation that can process local records without depending on a cloud database.

The application provides operational access for administrators, discipline officers, guidance counselors, teachers, and staff, subject to module permissions. Students are the subjects of managed records, while parents or guardians are recipients of notices and acknowledgment requests. Although the requirements identify possible student and parent access, a complete student or parent self-service portal was not established in the reviewed authentication and navigation implementation.

Table 1. Modules of eSIMS and their intended operational users

Module	Primary user	Documented function
Authentication and access	All authorized personnel	Sign in, establish a session, and determine accessible modules.
Operational dashboard	Authorized personnel	Display incident summaries and shortcuts to permitted workspaces.
Incident intake and evidence	Teacher / Staff; authorized officers	Record incidents, people involved, witnesses, and file references.
Risk classification rules	Administrator; authorized officers	Maintain keyword, location, and repeat-incident rule records.
Investigations	Discipline / Guidance; Administrator	Record findings, interviews, timelines, and follow-up issues.
Consequences and counseling	Authorized disciplinary personnel	Track assigned actions, schedules, notes, and completion.
Suspension and probation	Authorized disciplinary personnel	Manage orders, re-entry requirements, conditions, and progress.
Student information	Authorized personnel	Maintain student and guardian details; preview roster imports.
Behavioral profiles and action plans	Guidance; authorized personnel	Review conduct histories, merits, interventions, and milestones.
Parent notices and templates	Authorized personnel	Prepare letters, generate PDFs, and register notice records.
Acknowledgment tracking	Authorized personnel	Record receipt, dates, remarks, slips, and overdue follow-up.
Communication dispatch	Authorized personnel	Record simulated SMS and email dispatches for demonstration.
Reports and safety analytics	Administrator; authorized officers	Present location, recurrence, conduct, and executive summaries.
Audit and security views	Administrator; authorized personnel	Review activity and access records and diagnostic outputs.

Module	Primary user	Documented function
User and staff management	Administrator	Maintain accounts, role permissions, and staff directory records.
Settings and academic structure	Administrator	Maintain branding, school configuration, and maintenance settings.

Module access is configurable, and the roles in Table 1 identify normal operational responsibilities rather than an unconditional grant of access. The source includes both role checks and module-permission checks. In particular, the incident intake workflow distinguishes staff and teacher reporting from the authority to change official case status, risk, and resolution information.

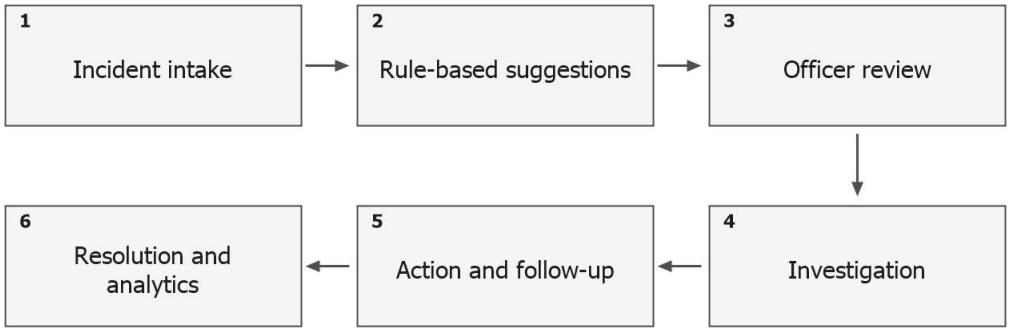


Figure 1. Overall incident-management workflow in eSIMS

Conceptual sequence derived from the implemented services; recommendations remain subject to authorized review.

Core entities contain synchronization identifiers and creation and update timestamps. Updating these entities can mark them as unsynchronized. These fields prepare records for future synchronization,

but they do not establish an operational cloud transfer service. Several supplementary tables are created through direct SQL, including communication logs, acknowledgment logs, and individual action-plan records; those tables do not all follow the same entity-base structure.

Implementation basis: SmartSchoolIncidentManagementSystem.csproj; App.xaml.cs; Data/AppDbContext.cs; Models/EntityBase.cs; Services/DatabaseInitializer.cs.

User Authentication and Role-Based Access Module

The authentication process accepts a username or email address and a password. It first searches the SystemUsers table and then provides a fallback to the Users table managed through Entity Framework Core. Successful authentication establishes the active user session and records a login activity. Logout clears the current session. Missing credentials and invalid credentials have separate feedback paths, and suspended or inactive accounts are checked in their respective account stores.

Registration collects account and role information, validates required fields, checks for an existing username, and stores a new active account. The current implementation does not require administrator approval before the newly registered account can sign in. This differs from the approval process in the sample scholarship document and is therefore described according to eSIMS itself.

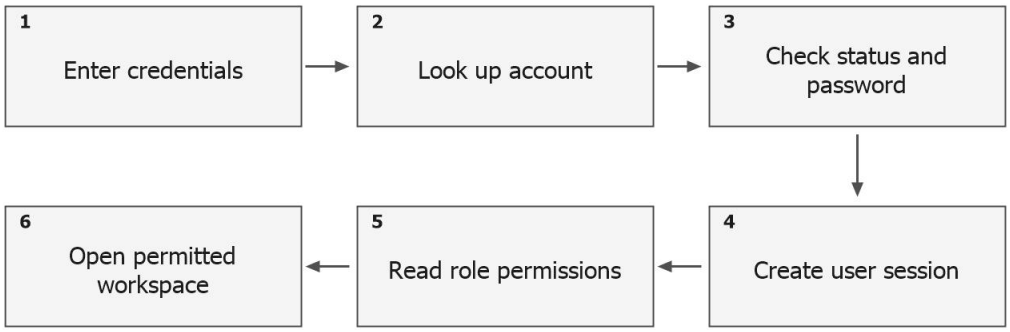


Figure 2. Authentication and permission workflow

Unsuccessful validation returns feedback to the login interface.

```
Function: AuthenticateUser
Require: Username or email and password
Ensure: An authenticated session or a validation message
1. Reject an empty username or password.
2. Search SystemUsers for the supplied username or email.
3. If found, check account status and validate the supplied
password.
4. If that lookup does not resolve the account, inspect the
Users fallback.
5. On success, assign the session identity and record the
login event.
6. Open the authenticated workspace; otherwise return an
error.
```

Pseudocode 1. Login and session establishment

This flow provides a common entry point for the application. However, the reviewed source also accepts legacy plaintext comparisons and demonstration password alternatives, while newly registered passwords use a SHA-256 digest. The existence of login validation must therefore be distinguished from a completed security assurance. Account approval, stronger password storage, removal of demonstration access,

and consistent enforcement across both user stores remain implementation recommendations.

```
Function: OpenAuthorizedModule
Require: An authenticated session and a selected module
Ensure: The module is opened only when the navigation checks
permit access
1. Read the current role from the active session.
2. Evaluate the module permissions stored in RolePermissions.
3. Use the documented default-role checks where the permission
lookup falls back.
4. Expose or open the relevant workspace when permission is
granted.
5. Apply action-level checks for restricted incident and
disciplinary changes.
```

Pseudocode 2. Permission-based module navigation

Navigation restrictions organize responsibilities, while action checks protect selected operations after a page is opened. The source explicitly restricts suspension and probation mutations through a shared disciplinary authorization check. Verification should still exercise direct actions and each relevant role rather than treating a hidden menu item as proof that every underlying operation is protected.

Implementation basis: Services/AuthService.cs; Services/SessionManager.cs; Services/DisciplinaryAuthorization.cs; ViewModels/LoginViewModel.cs; ViewModels/TabletDashboardViewModel.cs.

Operational Dashboard and Student Information Management

The dashboard presents an operational overview and entry points to incident reporting, investigations, behavioral work, parent notices,

reports, and configuration. Its available actions respond to the active role and module permissions. Summary cards and related navigation help the user move from a general caseload view to the records that require attention. The dashboard is a presentation of underlying records, not an independent source of measured school-safety outcomes.



Figure 3. Dashboard and record-management navigation

Available work areas depend on the current account and configured permissions.

```
Function: ViewOperationalDashboard
Require: An authenticated user with applicable module access
Ensure: Summary information and permitted module shortcuts
1. Load current incident data and session permissions.
2. Display the summary cards and available navigation actions.
3. Select an incident, investigation, behavior, notice, or
reporting action.
4. Transfer the applicable filter or selection to the target
workspace.
5. Refresh the displayed information after relevant changes.
```

Pseudocode 3. Dashboard review and quick navigation

Student information management provides a master record for names, student numbers, grade and section, guardian details, and related

profile information. Roster import supports CSV and XLSX files. Before insertion, the import workflow parses the file and displays a preview of valid and invalid rows. It checks imported identifiers against existing records and submits the valid rows through the student service. The reported success and failure counts allow staff to identify entries requiring correction.

```
Function: ImportStudentRoster
Require: A CSV or XLSX roster and access to student management
Ensure: Valid student rows submitted and import outcomes displayed
1. Select the roster file.
2. Load existing student identifiers for duplicate checks.
3. Parse the roster and show valid and invalid rows in the preview.
4. Allow confirmation only when valid rows are available.
5. Submit valid rows through the student service.
6. Display success and failure counts and refresh the student list.
```

Pseudocode 4. Student roster import with preview

A consistent student master list supports incident selection and subsequent conduct reporting. Some existing queries also match free-text student names, which can create ambiguity when names are similar or entered differently. The current model therefore benefits from selecting an existing student record, while the wider use of stable student identifiers remains a recommended improvement.

Implementation basis: ViewModels/DashboardViewModel.cs; ViewModels/DashboardViewModel.Caseload.cs; ViewModels/StudentsViewModel.Import.cs; Helpers/StudentImportHelper.cs; Services/StudentService.cs.

Incident Reporting and Evidence Management

Incident intake records an incident number, title, category, narrative, location, occurrence date and time, primary student, and reporting personnel. It also supports involved persons, witnesses, evidence references, notes, and immediate action information. The identifier follows the pattern INC-year-sequence. In the reviewed source, the sequence is calculated from the number of stored incidents; this implementation requires duplicate-identifier testing after deletions and simultaneous creation.

The intake interface refreshes its analysis when relevant narrative or student information changes. It displays suggested category, severity, risk, recommended action, and repeat-incident feedback. Staff and

teachers can report incidents and can update their own intake while it remains Reported. Official status, severity, risk, and resolution changes are reserved for the authorized administrative, discipline, or guidance workflow.

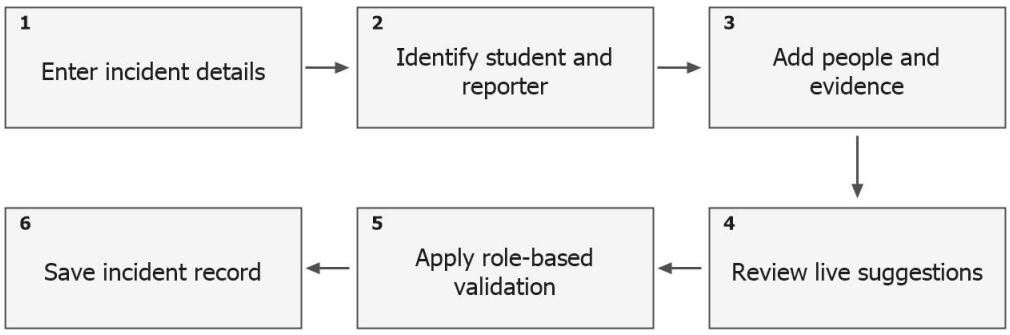


Figure 4. Incident intake and evidence association

Evidence metadata is linked to the incident; associated files are held in local application storage.

```
Function: SaveIncidentIntake
Require: Required incident details and permission for the selected operation
Ensure: An incident record with applicable metadata and analysis outputs
1. Validate the intake fields and identify the reporting user.
2. Match the selected student where a master record is available.
3. Serialize involved persons, witnesses, and evidence metadata.
4. For staff or teacher edits, require ownership and Reported status.
5. Apply the allowed status, risk, severity, and recommendation values.
6. Save through the incident service and refresh the record list.
```

Pseudocode 5. Incident creation and controlled editing

Involved persons, witness entries, and evidence metadata are serialized into fields associated with the incident. Selected evidence files are copied into local storage, and their paths support later access from the incident or investigation workspace. This provides a practical linkage between the narrative and its supporting materials. It should not be described as independently verified evidence authenticity or as encrypted storage without additional implementation and testing.

```
Function: AttachIncidentEvidence
Require: A selected local evidence file and an incident
context
Ensure: A local attachment reference associated with the
incident
1. Choose the evidence file and capture the available
descriptive details.
2. Create the incident attachment location when required.
3. Copy the selected file to the managed local evidence
folder.
4. Add its name, stored path, and metadata to the evidence
collection.
5. Serialize the collection when the incident is saved.
6. Use the saved reference when the record is reviewed.
```

Pseudocode 6. Evidence attachment processing

The persistence of an evidence reference does not by itself guarantee that the file remains available. A complete retention and recovery process must preserve the database together with its attachment folders. This is particularly relevant to parent acknowledgment slips and

investigative evidence because their database rows can remain present even if the associated files are moved or removed.

Implementation basis: Models/Incident.cs; Services/IncidentService.cs; ViewModels/IncidentListViewModel.Modal.cs; ViewModels/IncidentListViewModel.ModalSave.cs; ViewModels/IncidentListViewModel.Permissions.cs; ViewModels/IncidentListViewModel.InvolvedWitnessEvidence.cs.

Rule-Based Classification and Risk Assessment

Two related analysis implementations are present in eSIMS. The incident-entry workflow calls RuleEngineService for immediate suggestions. It scans the narrative with built-in keyword rules, considers sensitive location terms, and examines recent student incidents. A separate IncidentClassificationService supports configurable keyword weights, location multipliers, and repeat-offender rules. Its configurable analysis is available through the rule-maintenance functions, including the keyword test facility. The reviewed intake path does not call the configurable composite-scoring method.

The live engine raises severity and risk to the highest applicable keyword levels. A sensitive or poorly monitored location can raise a low risk to at least Medium. It examines incidents within the preceding 30 days: at least one matched prior incident raises a lower risk to Medium, while at least two raise a lower risk to High. These are the current source rules, rather than measured predictions of future behavior. Student

matching and the inclusion of the current record during editing should be checked during formal testing.



Figure 5. Live incident analysis and configurable rule analysis

The upper and lower sequences are distinct implementations; configuration changes are not established as controlling live intake.

Table 2. Risk levels and analysis rules observed in eSIMS

Level / rule	Live incident-analysis behavior	Configurable scoring behavior
Low	Default level when no higher trigger applies.	Score below 30, without immediate escalation.
Medium	Matching medium triggers; or a lower-risk case elevated by location or one prior incident.	Score from 30 to below 60, without immediate escalation.
High	Matching high triggers; or a lower-risk case with at least two matched prior incidents in 30 days.	Score from 60 to below 85, without immediate escalation.
Critical	Matching critical keyword triggers, including the configured built-in emergency terms.	Score at least 85, or an immediate-escalation condition.
Category	Retains a supplied specific category; can suggest a category from matched built-in terms.	Uses the highest-weight matched keyword category, then a supplied fallback.
Repeat behavior	Uses a fixed 30-day lookback in the live analysis.	Uses each active repeat rule's lookback and threshold.

```
Function: AnalyzeIncidentForIntake
Require: Narrative, location, student information, and
selected category
Ensure: Suggested classification, severity, risk, action, and
triggered rules
1. Initialize severity and risk to Low.
2. Scan the narrative for built-in keyword matches.
3. Retain the highest matched severity and risk values.
4. Apply the sensitive-location minimum risk where relevant.
5. Count matched student incidents within the 30-day lookback.
6. Apply repeat-incident risk escalation.
7. Return the suggestions, recommended response, and trigger
explanations.
```

Pseudocode 7. Live narrative-based incident assessment

The configurable engine calculates a separate score by summing matching keyword weights, multiplying the sum by the active location multiplier, and adding applicable repeat-rule points. The total is capped at 100. It supports exact-word, substring, and regular-expression matching. Restricted locations and selected rule settings can force immediate escalation. In the reviewed repeat-rule query, historical matching uses the student-name text and date cutoff; category-specific filtering is not applied to that count.

```
Function: EvaluateConfiguredRisk
Require: Narrative, optional location code, student text, and active rules
Ensure: A score, suggested category and severity, and an escalation flag
1. Sum the weights of active keyword rules matching the narrative.
2. Read the active location multiplier; otherwise use 1.0.
3. Round the product of keyword score and location multiplier.
4. Count historical matches for active repeat-rule lookback periods.
5. Add rule bonuses or set the immediate-escalation flag.
6. Cap the final score at 100.
7. Assign Low, Medium, High, or Critical using the score and escalation flag.
```

Pseudocode 8. Configurable composite-score calculation

For illustration only, a configured keyword sum of 20, a location multiplier of 1.5, and a repeat-rule bonus of 10 yield a score of 40, corresponding to Medium in this scoring method when no escalation flag applies. This arithmetic example explains the source formula and is not a test observation or a student assessment. The two engines should be reconciled before a claim is made that every rule edited by an administrator directly changes live incident classification.

Implementation basis: Services/RuleEngineService.cs; Services/IncidentClassificationService.cs; ViewModels/IncidentListViewModel.Modal.cs; ViewModels/KeywordMatrixViewModel.cs; ViewModels/LocationWeightingViewModel.cs; ViewModels/RepeatOffenderViewModel.cs.

Incident Lifecycle and Investigation Management

The implemented incident status enumeration contains Reported, UnderInvestigation, ActionTaken, Resolved, and Closed. Submitted and

Verified appear in the broader requirements but are not separate values in the current enumeration. The diagram below presents the intended progression using the implemented values. It does not imply that every transition is enforced as a single irreversible sequence.

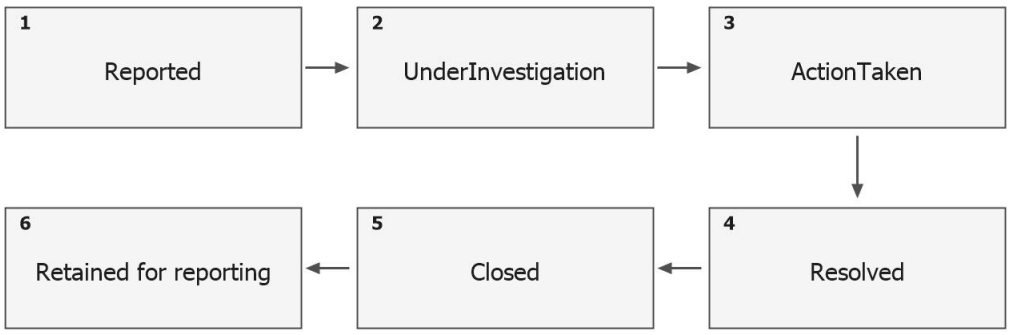


Figure 6. Implemented incident status sequence

The first five boxes show the implemented status values; the final box describes the use of retained records.

Opening an investigation through the service retrieves an existing investigation or creates one for the incident. A newly created record receives a start date, a target completion date seven days later, and Pending findings. An incident that is still Reported moves to UnderInvestigation. The investigation record supports a lead investigator name, interview logs, witness statements, notes, final findings, a summary, and structured timeline entries.

```
Function: ManageIncidentInvestigation
Require: An incident identifier and authorized investigation access
Ensure: An investigation linked to the incident with recorded findings and follow-up
1. Retrieve the investigation associated with the incident.
2. If none exists, create one with Pending findings and the default target date.
3. Move a Reported incident to UnderInvestigation.
4. Record interviews, witness information, evidence review, and timeline notes.
5. Flag missing or conflicting information for follow-up when required.
6. Save findings, summary, and follow-up resolution details.
```

Pseudocode 9. Investigation creation and documentation

Findings may be Pending, Substantiated, Unsubstantiated, Inconclusive, or Dismissed. These labels document the officer's determination after review; they are distinct from the automated risk suggestions. The follow-up flags allow incomplete information to remain visible while the officer gathers further evidence. When a disciplinary action is added, the service can move an incident from Reported or UnderInvestigation to ActionTaken. Completion of an action can optionally mark the related incident Resolved.

Implementation basis: Models/Enums.cs; Models/Investigation.cs; Services/InvestigationService.cs; ViewModels/InvestigationDetailViewModel.TimelineEvidence.cs; ViewModels/InvestigationDetailViewModel.ObservationsAndFlags.cs.

Disciplinary Actions, Restorative Interventions, and Probation

The disciplinary functions record the response to an incident and the follow-up required from the student or responsible officer. Consequence logs provide a record of the issued consequence and its current status. Counseling and other disciplinary actions record scheduling, assignment, details, completion, and follow-up notes. A recommended response is an input to officer review; it does not establish that the recommended action has actually been carried out.

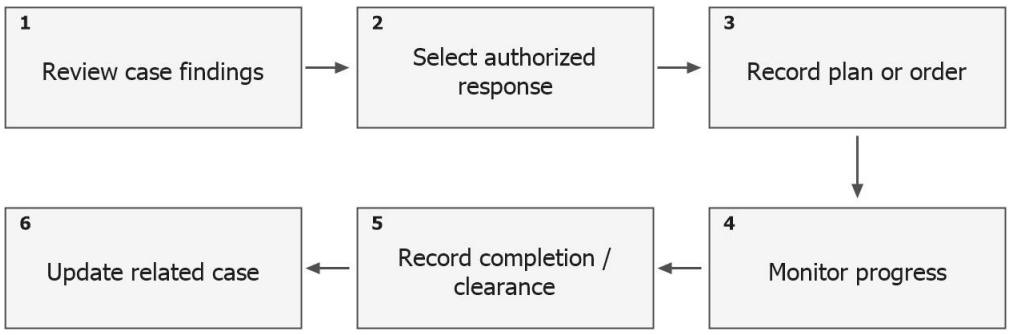


Figure 7. Action assignment and follow-up process

Different record types retain their own progress and clearance fields.

```
Function: RecordDisciplinaryResponse
Require: A reviewed incident and authority for the action
Ensure: A stored action with follow-up information and an
applicable case update
1. Select the incident and the appropriate response record
type.
2. Enter the student, responsible personnel, schedule, and
action details.
3. Apply the relevant permission checks.
4. Save the action and set applicable incident status to
ActionTaken.
5. Document progress, completion, or further follow-up.
6. Where supported and selected, resolve the incident after
action completion.
```

Pseudocode 10. Action assignment and case follow-up

Restorative plans track objectives, an assigned staff member, target hours or sessions, a start date, a deadline, and progress entries. Their completion percentage is calculated from the completed metric and target metric. This supports monitoring of activities such as counseling, mediation, and service. A recorded completion percentage describes progress against the entered target; it is not a measured treatment effect or proof that future incidents will cease.

Suspension records include dates, terms, a reason, an issuing officer, and re-entry requirements. The checklist can include a parent conference, an academic packet, and a counselor intake. The record distinguishes an active order, pending review, and readmission. Probation

records link the student and incident to expected behavior conditions, assigned staff, progress notes, extensions, and a final outcome summary.

```
Function: MonitorProbation
Require: A linked student and incident, conditions, dates, and
authorized personnel
Ensure: An active, extended, or completed probation record
with progress history
1. Verify disciplinary management permission.
2. Create the probation record and record the expected
behavior conditions.
3. Document progress notes and responsible staff observations.
4. If extending probation, record the revised period and
reason.
5. When completed, save the outcome summary and closure
information.
```

Pseudocode 11. Probation monitoring and completion

These modules extend the system beyond recording a violation. They preserve the tasks, dates, conditions, and observations needed for follow-up. The documentation establishes the presence of these tracking mechanisms, while their effectiveness and the correctness of their transitions remain subjects for scenario-based testing and school review.

Implementation basis: Services/ConsequenceService.cs; Services/RestorativeService.cs; Services/SuspensionService.cs; Services/ProbationService.cs; Models/RestorativePlan.cs; Models/SuspensionOrder.cs; Models/ProbationRecord.cs.

Student Behavioral Profiles and Individual Action Plans

The behavioral transcript consolidates incident entries, consequences, restorative interventions, suspension records, and merit awards for a selected registered student. It presents source references,

dates, score effects, handling personnel, and compliance information. A student who cannot be resolved from the student records does not receive a fabricated profile from the transcript service.

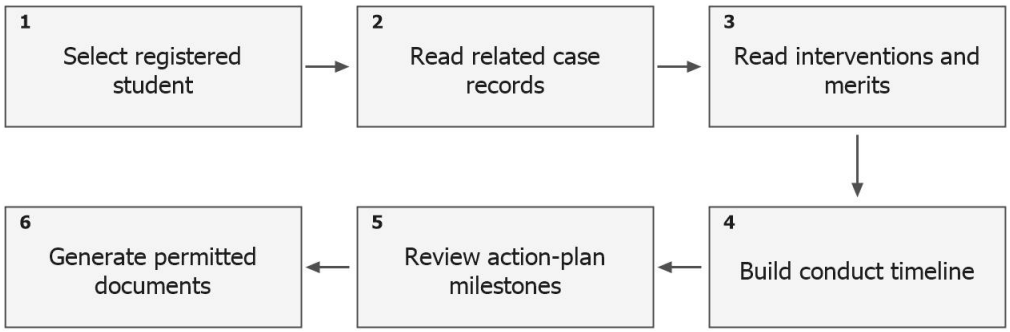


Figure 8. Student behavioral record consolidation

Anecdotal observations currently use a separate in-memory service.

```
Function: PrepareStudentConductTranscript
Require: An identifiable registered student
Ensure: A consolidated conduct record for review and supported PDF generation
1. Resolve the supplied student identifier or name against Students.
2. Return no transcript when no registered student is found.
3. Retrieve the related incident and disciplinary records.
4. Add intervention and merit entries with their recorded score effects.
5. Compute the current conduct result and arrange entries by date.
6. Display the profile and generate supported documents when requested.
```

Pseudocode 12. Behavioral transcript preparation

Individual action plans provide a more structured support record through milestones, counselor notes, and attachments. Their service

maintains dedicated SQLite tables for the plan and its related entries. The action-plan workspace can therefore document planned activities and subsequent observations alongside the broader incident history. It should be evaluated for correct student linkage and continuity between the displayed dossier and the stored records.

Anecdotal observations are handled differently. The reviewed AnecdotalService initializes sample records in a memory-resident list and performs its operations against that list. Consequently, this part of the current implementation should be treated as a demonstration workflow rather than confirmed durable storage across application restarts. Similarly, displayed intervention-success percentages returned as fixed sample values are not evidence of outcomes in the school.

Implementation basis: Services/BehavioralTranscriptService.cs; Services/ActionPlanService.cs; Services/AnecdotalService.cs; Services/RecidivismService.cs; ViewModels/BehaviorWorkspaceViewModel.cs.

Parent Notices, Letter Templates, and Communication Dispatch

Parent notices connect a selected student with a letter template and follow-up due date. The preparation workflow substitutes available student and officer information into the template and allows the notice text to be reviewed. The system creates an acknowledgment record for the notice and supports PDF generation. Letter templates can also be

maintained separately, allowing repeated school correspondence to use a consistent structure.

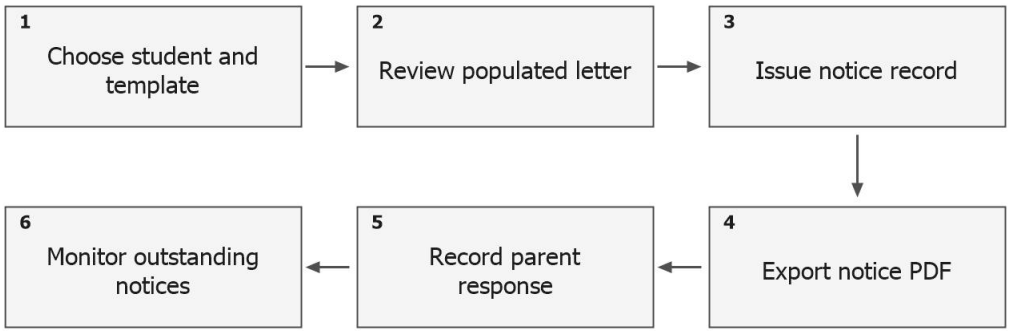


Figure 9. Parent-notice preparation and receipt tracking

Dispatch logging is available, but external SMS and email transport is simulated.

```
Function: PrepareParentNotice
Require: A selected student, notice details, template, and
relevant permission
Ensure: A notice acknowledgment record and a supported
printable notice
1. Select the student and letter template.
2. Populate the available template placeholders.
3. Review the subject, body, issue date, and due date.
4. Create the notice identifier and pending acknowledgment
record.
5. Save the notice record through the acknowledgment service.
6. Generate the notice PDF or proceed to the available
dispatch workflow.
```

Pseudocode 13. Parent-notice preparation

The communication service exposes SMS and email methods and writes dispatch records containing the recipient, channel, message, status, and time. In the reviewed implementation, these methods wait for a simulated delay and write the log; they do not call a live SMS gateway or

SMTP transport. The executive-summary email function follows the same pattern. A status such as Sent or Delivered in these demonstration logs must not be interpreted as independently confirmed receipt by a parent or school official.

This distinction preserves the practical contribution of the notice and receipt functions while identifying the work required for real delivery. Operational communication would require an actual transport integration, delivery or failure feedback, retry handling where appropriate, and tests confirming that the displayed status reflects the external service outcome.

Implementation basis: ViewModels/ParentNoticesViewModel.cs; ViewModels/ParentNoticesViewModel.Export.cs; Services/LetterTemplateService.cs; Services/CommunicationService.cs; Services/PdfExportService.cs.

Acknowledgment Tracking and Overdue Follow-Up

Acknowledgment tracking records the notice identifier, student name, issue date, due date, current status, date acknowledged, remarks, and attached file path. The tracking workspace includes search and status filters, together with counters for pending, overdue, and acknowledged records. A pending record is treated as overdue when its valid due date precedes the current date. This allows the displayed status to reflect the due date without requiring every overdue record to be manually changed first.

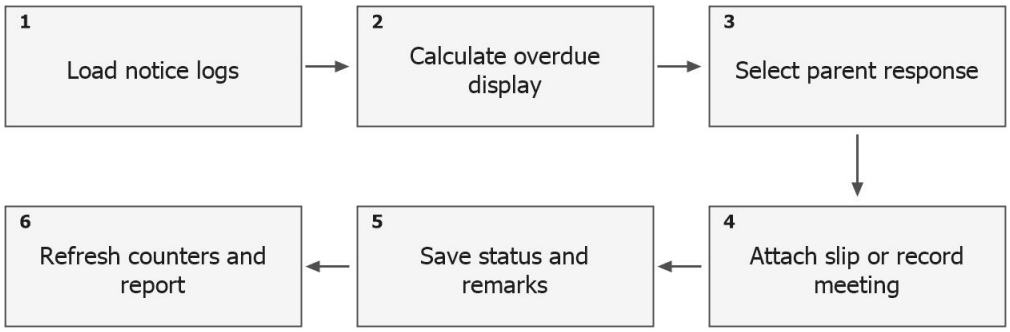


Figure 10. Acknowledgment recording and overdue monitoring

Signed slips and attended meetings map to Acknowledged; Non-Compliant remains a separate stored status.

```
Function: RecordAcknowledgment
Require: An existing notice record and its receipt or compliance details
Ensure: An updated acknowledgment log and refreshed tracking information
1. Select the notice to be updated.
2. Choose signed-slip acknowledgment, meeting attendance, or non-compliance.
3. Enter the acknowledgment date and remarks.
4. If a slip is supplied, copy it into local Scanned_Slips storage.
5. Map either acknowledgment option to Acknowledged; otherwise use Non-Compliant.
6. Update the record by its identifier through parameterized SQL.
7. Reload the logs, filters, and summary counters.
```

Pseudocode 14. Recording parent acknowledgment

The service uses parameterized insert, update, and delete commands for acknowledgment data. It can therefore maintain receipt information independently of the PDF-generation process. The interface offers an overdue compliance report when outstanding records are

available. Its status-filter list includes All, Pending, Overdue, and Acknowledged; Non-Compliant is stored but has no dedicated filter in the reviewed tracking view model.

This module provides the closing step between issuing a notice and recording the family's response. A receipt is a documented administrative event rather than proof that an external message was delivered automatically. The source also initializes sample acknowledgment records when its table is empty. Demonstration entries must be distinguished from actual school records when preparing operational reports or evaluating follow-up performance.

Implementation basis: Services/AcknowledgmentService.cs; Models/AcknowledgmentModels.cs; ViewModels/AcknowledgmentTrackingViewModel.cs; Services/PdfExportService.Reports.cs.

Reports, Analytics, and PDF Generation

The reporting modules present incident and conduct information in forms suitable for administrative review. The implementation includes campus location summaries, recurrence analysis, conduct summaries, and executive reports. Date and category filters allow the location analysis to focus on a selected period and incident class. The recurrence service distinguishes students associated with one, two, or at least three incidents

in its summary calculation, while other repeat-alert views use their own query logic.

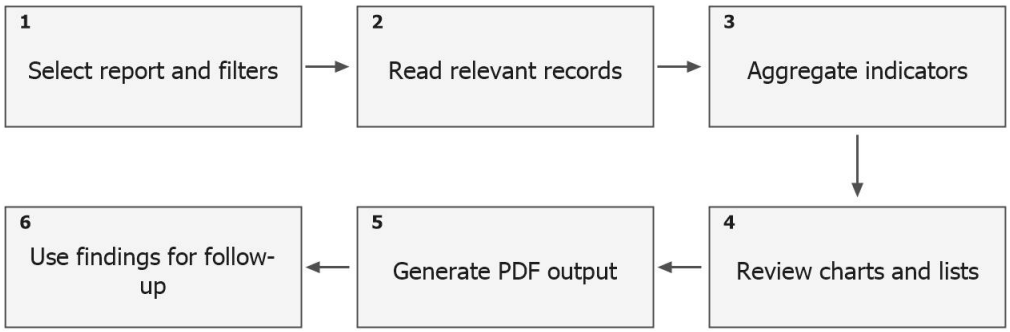


Figure 11. Administrative reporting workflow

Fixed demonstration metrics must be excluded from claims about actual school outcomes.

```
Function: GenerateAdministrativeReport
Require: A report selection, applicable filters, and accessible records
Ensure: A displayed summary or a generated PDF file
1. Choose the reporting area and its available timeframe or category filters.
2. Retrieve the relevant incident, conduct, or acknowledgment data.
3. Calculate the supported summary values and prepare the display.
4. Review the records represented by the figures.
5. Choose a destination and generate the requested PDF.
6. Treat executive email dispatch as simulated until transport is integrated.
```

Pseudocode 15. Filtered administrative report preparation

Supported PDF outputs include conduct transcripts, good-moral certificates, parent notices, overdue compliance reports, campus safety reports, recurrence reports, executive summaries, audit ledgers, security-

access reports, and diagnostic certificates. Their presence is established by the export-service methods. The review did not establish a general Excel report-export implementation, although the student-import workflow does accept XLSX files. Import capability should therefore not be presented as equivalent to report export.

The reporting functions can make stored incidents easier to summarize and retrieve. They do not establish a reduction in incidents, improved response time, or intervention effectiveness without comparative measurements. In particular, fixed intervention-success values in the recurrence service and seeded records must be identified as demonstrations when results are interpreted.

Implementation basis: Services/AnalyticsService.cs; Services/HeatmapService.cs; Services/RecidivismService.cs; Services/PdfExportService.cs; Services/PdfExportService.Reports.cs; Services/PdfExportService.Security.cs; ViewModels/ReportsAnalyticsViewModel.Export.cs.

Notifications, Audit Trail, and System Configuration

The notification service prepares in-application items from recent incident information, including reported cases, higher-risk incidents, and counseling-related entries. It maintains its notification collection in memory and notifies the interface when the collection changes. This implementation provides a local notification experience, but it does not establish a durable external messaging or push-notification service.

The audit functions record and display selected activities, while the security-access service provides storage and retrieval of access-event records. These are useful foundations for accountability. Nevertheless, a claim that every critical action is logged requires testing of each action path. The integrity diagnostic also needs careful interpretation: its row-hash routine computes a hash from current incident values and checks that a hash was produced; it does not compare those values with a previously stored trusted signature.

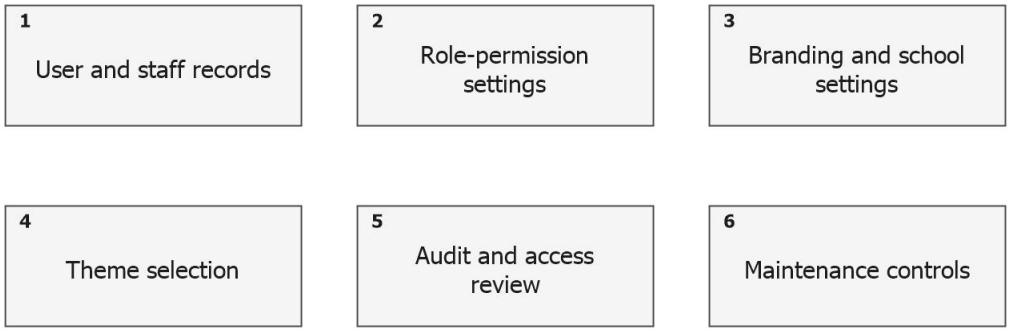


Figure 12. Administrative configuration and review areas

This figure groups administrative functions; it does not represent a single transaction or verified backup process.

```
Function: MaintainSchoolConfiguration
Require: Administrative access and configuration values
Ensure: Stored settings and refreshed relevant interface
information
1. Open the permitted settings or administration workspace.
2. Edit the school, branding, academic, or account
configuration.
3. Validate the information required by the selected
operation.
4. Save through the applicable service or data-access path.
5. Refresh dependent branding, permissions, or display state.
6. Review available activity records for the operation.
```

Pseudocode 16. Administrative settings maintenance

ThemeService switches between Light and Dark resource dictionaries and updates application resources. The window also exposes minimize and close controls. These features support the intended kiosk-style presentation and user preference for interface brightness. Their presence does not replace a visual accessibility review of every control under both themes or a deployment test on the target screen sizes.

The settings workspace includes a database-backup command, but its source path differs from the active database configuration. AppDbContext and DatabaseInitializer use the SmartSchool folder with incidents.db, whereas the backup command targets SmartSchoolIncidentManagement with school_incidents.db. The command can proceed to a success message even when the expected source file does not exist. A reliable backup and restore capability cannot be claimed

from this implementation. No functioning remote synchronization procedure was established in the reviewed workflow.

Implementation basis: Services/NotificationService.cs; Services/AuditLogger.cs; Services/IntegrityAuditService.cs; Services/SecurityAccessLogService.cs; Services/ThemeService.cs; ViewModels/SettingsWorkspaceViewModel.cs; MainWindow.xaml.cs.

Functional Verification Results

For this documentation, functional review consisted of tracing the source-level inputs, processing paths, and stored or displayed outputs. The application was not modified or exercised against its operational database. Consequently, Table 3 reports implementation evidence and known limitations rather than assigning runtime pass or fail results. Formal test outcomes supplied by the researcher should be recorded separately with the test date, environment, input, expected result, actual result, and supporting evidence.

Table 3. Functional review summary and formal testing status

Verification area	Source-review finding	Formal result
Authentication and roles	Login, account status checks, session management, and permissions are present. Demonstration access alternatives require attention.	Pending
Student management and import	Student records and CSV/XLSX preview-and-import paths are present.	Pending
Incident intake	Creation, editing restrictions, and evidence metadata paths are present.	Pending

Verification area	Source-review finding	Formal result
Live rule suggestions	Built-in narrative, location, and recent-history logic is called from intake.	Pending
Configurable rule engine	Rule maintenance and weighted scoring are present; live-intake integration is not established.	Pending
Investigation lifecycle	Investigation creation, findings, follow-up, and selected status updates are present.	Pending
Disciplinary follow-up	Actions, restorative plans, suspension, and probation services are present.	Pending
Parent notices and receipts	Notice creation, local slips, acknowledgment updates, and overdue reports are present.	Pending
External SMS and email	Methods simulate transport and write dispatch logs.	Integration required
Anecdotal record persistence	The service operates on an in-memory sample list.	Persistence required
Analytics and exports	Aggregation and PDF methods are present; some success metrics are fixed sample values.	Pending
Integrity verification	Current-row hashes are computed without trusted baseline comparison.	Correction required
Backup and restore	Backup source differs from the active database; recovery is not established.	Correction required
Remote synchronization	Sync metadata exists; an operational remote-sync workflow was not established.	Deferred
Theme and kiosk presentation	Theme switching and window controls are present; visual checks are still required.	Pending

The review identifies substantial implementation across the intended incident-management lifecycle. It also identifies boundaries that directly affect a deployment or evaluation claim. Most notably, live rule evaluation and editable scoring are separate paths, communication delivery is simulated, anecdotal persistence is incomplete, and backup and

integrity outputs need correction. These findings should guide formal verification rather than be converted into assumed successful tests.

A suitable execution record should cover valid and invalid login; each role's allowed and denied actions; required-field validation; duplicate and invalid roster rows; narrative and risk boundary cases; repeated-incident counting; investigation transitions; suspension and probation permission checks; due-date boundaries; missing attachment files; restart persistence; and report totals reconciled with selected records. Test timing and supported-machine details are necessary before performance or portability conclusions can be reported.

Evaluation of the System Using ISO/IEC 25010

To preserve the structure of the supplied guide, this draft organizes the evaluation around the eight product-quality characteristics associated with ISO/IEC 25010:2011: functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability. The official ISO catalogue distinguishes this eight-characteristic model from ISO/IEC 25010:2023, whose product-quality model contains nine characteristics. The edition used in the final study must agree with the approved methodology.

Standards basis: ISO/IEC 25010:2011, https://www.iso.org/standard/35733.html; ISO/IEC 25010:2023, https://www.iso.org/standard/78176.html.

The numerical tables that follow contain illustrative simulated ratings, constructed at the researcher's request to demonstrate the layout, calculations, and discussion of an evaluation. They are not collected responses, validated questionnaire results, expert assessments, or scores inferred from code inspection. No respondent sample size is asserted. The indicative strengths and limitations discussed elsewhere are source-review findings and remain separate from these invented numbers.

The example role columns use A for administrators, D/G for discipline officers and guidance counselors, T/S for teachers and staff, and IT for information technology evaluators. These are hypothetical

evaluation categories, not groups that participated in a survey. Maintainability uses an IT-only example. Every numerical cell in Tables 5 through 13 is simulated.

Table 4. Five-point interpretation ranges reproduced from the supplied guide

Scale	Mean range	Verbal interpretation
5	4.50–5.00	Strongly Agree
4	3.50–4.49	Agree
3	2.50–3.49	Neither Agree nor Disagree
2	1.50–2.49	Disagree
1	1.00–1.49	Strongly Disagree

These ranges reproduce the guide's convention and are not asserted to be the approved eSIMS instrument. They should be confirmed against Chapter III and the actual questionnaire. Ratings are descriptive judgments about the evaluated version and task; they should not be used as substitutes for objective security tests, timed performance measures, or data-recovery verification.

$$\textbf{Item mean} = \Sigma(f \times x) / N$$

For actual survey data, f is the response frequency, x is the scale value, and N is the valid response count for that item. In the illustrative tables, no raw responses or respondent counts exist. Instead, four hypothetical group means are assigned to each item; their equally

weighted average gives its illustrative mean. A characteristic mean averages its four unrounded item means. The overall example averages the eight unrounded characteristic means. Maintainability uses its four IT-only values. Reported values are rounded to two decimals; actual research should use the aggregation method approved in Chapter III.

Illustrative Simulated Evaluation in Terms of Functional Suitability

Functional suitability concerns the extent to which the available functions support the intended school tasks. The indicators address incident records, lifecycle support, review assistance, and parent follow-up. Respondents should evaluate functions they actually used, while unintegrated communication and rule paths should be identified in the evaluation conditions.

Table 5. Illustrative simulated ratings for Functional Suitability (not respondent data)

Indicator	A	D/G	T/S	IT	Mean	Verbal rating
The system records the information needed to document a school incident.	4.40	4.30	4.20	4.10	4.25	Agree
The incident, investigation, and follow-up functions support my assigned tasks.	4.20	4.20	4.10	3.90	4.10	Agree
Risk suggestions and their explanations assist authorized case review.	3.90	3.80	3.80	3.50	3.75	Agree
Notice and acknowledgment records support parent follow-up.	4.30	4.20	4.10	4.00	4.15	Agree
Simulated consolidated mean	4.20	4.13	4.05	3.88	4.06	Agree

Illustrative simulated data only. No respondents were surveyed for these values; equal weighting is used solely for this numerical example.

The simulated consolidated mean is 4.06 (Agree). The highest illustrative item mean is 4.25 and the lowest is 3.75. These constructed values show how an evaluation would be reported; they do not establish actual user experience or system quality. Replace them with the validated results before reporting empirical findings.

Illustrative Simulated Evaluation in Terms of Performance Efficiency

Performance efficiency requires both user feedback and timed measurements on an identified machine and data volume. The present source review does not supply response-time measurements. The

approved test protocol must define acceptable times and workload before the indicators are interpreted.

Table 6. Illustrative simulated ratings for Performance Efficiency (not respondent data)

Indicator	A	D/G	T/S	IT	Mean	Verbal rating
Incident lists and student searches respond within the approved time limit.	4.10	4.00	4.10	3.90	4.03	Agree
Saving an incident and refreshing its analysis respond within the approved time limit.	4.00	3.90	4.00	3.80	3.93	Agree
Filtered summaries and PDF reports complete within the approved time limit.	3.90	3.80	3.90	3.70	3.83	Agree
The application remains responsive with the agreed test data volume.	3.80	3.70	3.80	3.60	3.73	Agree
Simulated consolidated mean	3.95	3.85	3.95	3.75	3.88	Agree

Illustrative simulated data only. No respondents were surveyed for these values; equal weighting is used solely for this numerical example.

The simulated consolidated mean is 3.88 (Agree). The highest illustrative item mean is 4.03 and the lowest is 3.73. These constructed values show how an evaluation would be reported; they do not establish actual user experience or system quality. Replace them with the validated results before reporting empirical findings.

Illustrative Simulated Evaluation in Terms of Compatibility

Compatibility is evaluated against the supported Windows environment, roster-import formats, PDF output, and local file references. These indicators do not assume MySQL, a web browser client, or a remote database, since those belong to a different system or are outside the established implementation.

Table 7. Illustrative simulated ratings for Compatibility (not respondent data)

Indicator	A	D/G	T/S	IT	Mean	Verbal rating
The application operates with the required software on the target Windows workstation.	4.20	4.10	4.10	4.00	4.10	Agree
Supported CSV and XLSX roster files are imported with correct field mapping.	4.10	4.00	4.00	3.90	4.00	Agree
Generated PDF documents open and display correctly in the chosen viewer.	4.30	4.20	4.20	4.10	4.20	Agree
Attached evidence and acknowledgment files can be retrieved from their saved locations.	4.00	3.90	3.90	3.70	3.88	Agree
Simulated consolidated mean	4.15	4.05	4.05	3.93	4.04	Agree

Illustrative simulated data only. No respondents were surveyed for these values; equal weighting is used solely for this numerical example.

The simulated consolidated mean is 4.04 (Agree). The highest illustrative item mean is 4.20 and the lowest is 3.88. These constructed values show how an evaluation would be reported; they do not establish

actual user experience or system quality. Replace them with the validated results before reporting empirical findings.

Illustrative Simulated Evaluation in Terms of Usability

Usability focuses on how the intended personnel understand and complete their tasks. Evaluation should include first-use navigation, correcting invalid input, finding a case, and recording an acknowledgment. Light and Dark modes should be examined on the target display rather than rated from the presence of a theme-switch command alone.

Table 8. Illustrative simulated ratings for Usability (not respondent data)

Indicator	A	D/G	T/S	IT	Mean	Verbal rating
The navigation makes my permitted tasks easy to locate.	4.40	4.30	4.40	4.20	4.33	Agree
Incident labels, instructions, and validation messages are understandable.	4.20	4.10	4.20	4.00	4.13	Agree
The main tasks can be completed with a reasonable amount of training.	4.30	4.20	4.30	4.10	4.23	Agree
Text, controls, and status information remain readable in the selected theme.	4.20	4.10	4.20	4.00	4.13	Agree
Simulated consolidated mean	4.28	4.18	4.28	4.08	4.20	Agree

Illustrative simulated data only. No respondents were surveyed for these values; equal weighting is used solely for this numerical example.

The simulated consolidated mean is 4.20 (Agree). The highest illustrative item mean is 4.33 and the lowest is 4.13. These constructed values show how an evaluation would be reported; they do not establish actual user experience or system quality. Replace them with the validated results before reporting empirical findings.

Illustrative Simulated Evaluation in Terms of Reliability

Reliability concerns repeatable operation and retention of information. The in-memory anecdotal service and the backup-path discrepancy are specific review findings relevant to these indicators. Backup recovery and restart persistence must be demonstrated before favorable reliability conclusions are drawn.

Table 9. Illustrative simulated ratings for Reliability (not respondent data)

Indicator	A	D/G	T/S	IT	Mean	Verbal rating
Saved operational records remain available after restarting the application.	3.60	3.50	3.50	3.20	3.45	Neither Agree nor Disagree
Updating a case preserves the consistency of related records.	3.70	3.60	3.50	3.20	3.50	Agree
Errors and missing attachment files are handled without losing saved work.	3.50	3.40	3.40	3.00	3.33	Neither Agree nor Disagree
A verified backup can restore the required records and associated files.	2.70	2.60	2.60	2.20	2.53	Neither Agree nor Disagree
Simulated consolidated mean	3.38	3.28	3.25	2.90	3.20	Neither Agree nor Disagree

Illustrative simulated data only. No respondents were surveyed for these values; equal weighting is used solely for this numerical example.

The simulated consolidated mean is 3.20 (Neither Agree nor Disagree). The highest illustrative item mean is 3.50 and the lowest is 2.53. These constructed values show how an evaluation would be reported; they do not establish actual user experience or system quality. Replace them with the validated results before reporting empirical findings.

Illustrative Simulated Evaluation in Terms of Security

Security evaluation should combine appropriate personnel feedback with technical checks. The current demonstration login alternatives, registration behavior, mixed account stores, and integrity-check limitations

require attention. User agreement with a questionnaire does not prove resistance to unauthorized access or establish that every sensitive operation is audited.

Table 10. Illustrative simulated ratings for Security (not respondent data)

Indicator	A	D/G	T/S	IT	Mean	Verbal rating
Users can access only the modules and records permitted for their role.	3.80	3.70	3.60	3.20	3.58	Agree
Restricted case and disciplinary changes are denied to unauthorized users.	3.90	3.80	3.70	3.40	3.70	Agree
Account credentials and sensitive records meet the approved protection requirements.	2.50	2.40	2.50	2.00	2.35	Disagree
Activity records allow relevant actions to be traced to responsible users.	3.50	3.40	3.30	2.80	3.25	Neither Agree nor Disagree
Simulated consolidated mean	3.43	3.33	3.28	2.85	3.22	Neither Agree nor Disagree

Illustrative simulated data only. No respondents were surveyed for these values; equal weighting is used solely for this numerical example.

The simulated consolidated mean is 3.22 (Neither Agree nor Disagree). The highest illustrative item mean is 3.70 and the lowest is 2.35. These constructed values show how an evaluation would be reported; they do not establish actual user experience or system quality. Replace them with the validated results before reporting empirical findings.

Illustrative Simulated Evaluation in Terms of Maintainability

Maintainability is intended for qualified IT evaluators who inspect the code and attempt representative changes in a controlled environment. The modular structure is useful evidence, but separate rule engines, direct-SQL tables, shared-context usage, and silent exception handling should be considered in a technical assessment.

Table 11. Illustrative simulated ratings for Maintainability (not respondent data)

Indicator	IT mean	Verbal rating
The separation of models, services, repositories, and view models supports modification.	4.10	Agree
Errors can be diagnosed from the available validation and logging information.	3.40	Neither Agree nor Disagree
Rule changes and related behavior can be tested without changing unrelated modules.	3.30	Neither Agree nor Disagree
Database structure and configuration can be maintained consistently.	3.50	Agree
Simulated consolidated mean	3.58	Agree

Illustrative simulated data only. No respondents were surveyed for these values; equal weighting is used solely for this numerical example.

The simulated consolidated mean is 3.58 (Agree). The highest illustrative item mean is 4.10 and the lowest is 3.30. These constructed values show how an evaluation would be reported; they do not establish actual user experience or system quality. Replace them with the validated results before reporting empirical findings.

Illustrative Simulated Evaluation in Terms of Portability

Portability is limited here to installation and transfer within the approved Windows desktop environment. The project target alone does not establish successful operation on every Windows version or screen resolution. This section does not claim native support for mobile, browser, macOS, or Linux clients.

Table 12. Illustrative simulated ratings for Portability (not respondent data)

Indicator	A	D/G	T/S	IT	Mean	Verbal rating
The application installs and starts on each approved Windows target.	3.90	3.80	3.80	3.70	3.80	Agree
The interface remains usable at the approved display sizes and scaling settings.	3.90	3.80	3.90	3.70	3.83	Agree
Required files and database locations are configured correctly on a new workstation.	3.80	3.70	3.70	3.50	3.68	Agree
A documented transfer procedure preserves the database and required attachments.	3.20	3.10	3.10	2.90	3.08	Neither Agree nor Disagree
Simulated consolidated mean	3.70	3.60	3.63	3.45	3.59	Agree

Illustrative simulated data only. No respondents were surveyed for these values; equal weighting is used solely for this numerical example.

The simulated consolidated mean is 3.59 (Agree). The highest illustrative item mean is 3.83 and the lowest is 3.08. These constructed values show how an evaluation would be reported; they do not establish

actual user experience or system quality. Replace them with the validated results before reporting empirical findings.

Summary of Illustrative Simulated Evaluation Ratings

Table 13. Summary of illustrative simulated ratings (not respondent data)

Quality characteristic	Mean	Descriptive equivalent
Functional Suitability	4.06	Agree
Performance Efficiency	3.88	Agree
Compatibility	4.04	Agree
Usability	4.20	Agree
Reliability	3.20	Neither Agree nor Disagree
Security	3.22	Neither Agree nor Disagree
Maintainability	3.58	Agree
Portability	3.59	Agree
Illustrative overall mean	3.72	Agree

Illustrative simulated data only. No respondents were surveyed for these values; equal weighting is used solely for this numerical example.

The arithmetic example gives an overall simulated mean of 3.72, described as Agree using the guide's ranges. In this invented dataset, usability has the highest characteristic mean and reliability has the lowest. This ordering is a feature of the constructed example and must not be reported as an observed finding. No overall acceptance conclusion is established. Actual evaluation must identify the questionnaire, participant groups and counts, application version, and test environment.

Chapter Synthesis

The reviewed eSIMS implementation brings together incident records, live rule suggestions, investigations, disciplinary and restorative follow-up, student conduct information, parent notices, acknowledgment tracking, and administrative reporting. Its local SQLite data layer and modular desktop structure provide the basis for a coordinated school incident workflow. The acknowledgment function is a particularly clear link between an issued notice and the recorded response, including dates, remarks, local slips, and overdue follow-up.

The development findings also delimit the claims that can be made. Editable scoring and live intake use separate analysis paths; SMS and email dispatch are simulated; anecdotal records are not durably stored by their current service; some displayed metrics are demonstrations; and backup and integrity verification need further work. These are source-review findings, not formal pass/fail outcomes. The accompanying numerical evaluation is explicitly simulated and does not supply empirical evidence of acceptance or effectiveness.

CHAPTER V

SUMMARY, CONCLUSIONS, AND RECOMMENDATIONS

Summary

This documentation examined eSIMS: Smart School Incident Management System Using Rule-Based Algorithms for Incident Classification and Risk Assessment. The application is designed to support the recording, analysis, investigation, and follow-up of school incidents through a Windows desktop interface. It uses C#, WPF, the .NET 8 target framework, and a local SQLite database, with service abstractions and view models separating many of the data-processing and presentation responsibilities.

The implemented functions cover incident intake, evidence references, rule-based suggestions, investigation records, disciplinary actions, restorative plans, suspension and probation tracking, student information, behavioral transcripts, individual action plans, parent notices, acknowledgment records, and PDF reporting. Administrative functions include accounts, role permissions, staff records, school settings, themes, and audit-related workspaces. The operational roles represented in the

reviewed source are administrators, discipline officers, guidance counselors, teachers, and staff.

Incident entry uses built-in rules to provide immediate suggestions, while a separate configurable service maintains weighted keyword, location, and repeated-incident rules. The incident lifecycle uses Reported, UnderInvestigation, ActionTaken, Resolved, and Closed. Investigations and assigned responses provide records of officer review and follow-up. Parent acknowledgment tracking records receipt details and identifies overdue notices using stored dates and status information.

The source review established substantial module coverage but also identified limits in the reviewed version. External dispatch is simulated; anecdotal records use an in-memory list; some analytics values are fixed demonstration data; configurable scoring is separate from live intake; and the backup and integrity routines do not establish reliable recovery or tamper detection. The project contains synchronization metadata, but an operational remote-sync workflow was not established.

The evaluation layout follows the supplied chapter guide, with functional review and eight software-quality sections. At the researcher's request, illustrative group ratings and calculations were constructed to demonstrate the presentation. Their illustrative overall mean is 3.72,

described as Agree under the reproduced scale. This is not an actual survey result. No respondent count, observed acceptance level, runtime pass rate, or measured improvement in school operations was established.

Conclusions

Based on the reviewed implementation, eSIMS provides a substantial foundation for organizing school incident information and the related administrative follow-up in one desktop application. Its principal contribution is the connection of incident details with analysis suggestions, investigation records, response tracking, student histories, parent notices, and report generation. The presence of these modules supports the conclusion that the development has addressed major parts of the intended workflow.

The rule-based functions provide explainable triggers and suggested levels that can assist authorized personnel in reviewing a case. They should remain decision-support mechanisms, with findings and disciplinary decisions recorded by responsible officers. Because the live and configurable analysis paths are separate, the current implementation does not support an unrestricted claim that editing a rule automatically changes every incident recommendation.

Acknowledgment tracking supports administrative accountability by linking a notice to the recorded response, date, remarks, and attached slip. It also provides a basis for identifying outstanding follow-up. This contribution is distinct from external message delivery, which is simulated in the reviewed communication service. Similarly, a displayed compliance or integrity status is not sufficient evidence of successful recovery, authentic evidence, or protection from unauthorized changes.

A firm conclusion that all system objectives have been achieved, that the application is ready for operational deployment, or that intended users have accepted it requires actual functional tests and validated evaluation results. The current findings justify continued verification and focused improvement. They do not establish statistical user satisfaction, improved response time, reduced incident frequency, or complete reliability and security.

Recommendations

The following recommendations follow directly from the documented implementation and the evidence still required for final evaluation. They are proposed development and verification work; they were not applied to the application during preparation of this document.

1. Complete the empirical evaluation. Replace every simulated numerical value and its interpretation with actual respondent groups, valid response counts, item ratings or frequencies, and formal test outcomes. Confirm the questionnaire, interpretation ranges, quality-model edition, and aggregation method against the approved methodology. Do not present the illustrative ratings as collected research evidence.

2. Unify rule execution. Connect live incident suggestions and configurable rule management to a clearly defined analysis path. Test whether changes to keyword weights, location rules, and recurrence thresholds produce the intended changes in intake. Address student-name matching, category handling, and the counting of the current incident during editing.

3. Strengthen account and permission handling. Remove demonstration password alternatives and legacy plaintext comparisons, adopt an appropriate password-storage scheme, and review registration and role assignment. Verify each sensitive action for every relevant role and reconcile the SystemUsers and Users account paths.

4. Correct backup and verify restoration. Make the backup source agree with the database actually used by the application. Report missing or failed copies accurately, use a database-consistent backup

procedure, preserve attachment folders, and demonstrate restoration in an isolated test environment before claiming recoverability.

5. Complete durable storage and data consistency. Persist anecdotal observations in the intended data layer. Review the direct-SQL tables for consistent identifiers, relationships, timestamps, and update behavior. Prefer stable student identifiers over partial-name matching, and test record associations after import, editing, and deletion.

6. Replace simulated communication with verified delivery. Integrate the selected SMS or email transport before operational use. Distinguish queued, submitted, delivered, and failed states where supported, and validate them against external service responses. Keep notice issuance and parent acknowledgment as separately recorded events.

7. Improve audit and integrity assurance. Verify the coverage of activity records and avoid presenting a newly computed hash as proof of unchanged data. Establish a trusted comparison or other appropriate integrity mechanism if tamper detection is a requirement. Preserve errors needed to diagnose unsuccessful checks.

8. Separate demonstration values from operational reporting. Remove or clearly identify seeded records and fixed

intervention-success metrics in production reporting. Compute school-specific measures from defined records and denominators, and reconcile PDF totals with the corresponding filtered data.

9. Validate the complete incident and acknowledgment lifecycle. Test case transitions, action completion, suspension re-entry, probation extension and closure, overdue date boundaries, non-compliant status, and missing slip files. Confirm that the displayed counts and filters agree with the stored records.

10. Verify installation, usability, and accessibility. Test the agreed Windows targets, display scaling, both themes, keyboard use, and common task errors. Use these observations with user feedback to improve navigation and form clarity. Add student or parent self-service and remote synchronization only when those extensions are explicitly implemented and evaluated.

Documentation Sources

The primary implementation evidence for these chapters is the local eSIMS project reviewed on 18 October 2026. The supplied eSkolar chapter document was used as a formatting and organizational reference. Its scholarship functions, respondent data, screenshots, and conclusions

were not treated as eSIMS evidence. The numerical examples in this document are separately constructed simulations, explicitly labeled wherever reported.

Project scope and architecture: SYSTEM_REQUIREMENTS.md; SmartSchoolIncidentManagementSystem.csproj; App.xaml.cs; Data/AppDbContext.cs; Services/DatabaseInitializer.cs.

Authentication and access: Services/AuthService.cs; Services/SessionManager.cs; Services/DisciplinaryAuthorization.cs; ViewModels/LoginViewModel.cs; ViewModels/TabletDashboardViewModel.cs.

Incidents and analysis: Models/Incident.cs; Models/Enums.cs; Services/IncidentService.cs; Services/RuleEngineService.cs; Services/IncidentClassificationService.cs; the IncidentListViewModel partial files.

Investigation and follow-up: Services/InvestigationService.cs; Services/ConsequenceService.cs; Services/RestorativeService.cs; Services/SuspensionService.cs; Services/ProbationService.cs; their related models and view models.

Students and behavior: Services/StudentService.cs; Helpers/StudentImportHelper.cs; Services/BehavioralTranscriptService.cs; Services/ActionPlanService.cs; Services/AnecdotalService.cs; ViewModels/StudentsViewModel.Import.cs.

Notices and receipts: Services/LetterTemplateService.cs; Services/CommunicationService.cs; Services/AcknowledgmentService.cs; Models/AcknowledgmentModels.cs; ViewModels/AcknowledgmentTrackingViewModel.cs; the ParentNoticesViewModel partial files.

Reporting and administration: Services/AnalyticsService.cs; Services/HeatmapService.cs; Services/RecidivismService.cs; the PdfExportService partial files; Services/NotificationService.cs; Services/IntegrityAuditService.cs; Services/SecurityAccessLogService.cs; Services/ThemeService.cs; ViewModels/SettingsWorkspaceViewModel.cs.

Formatting reference: eskolar_chapter_4-5 (final).docx, supplied by the researcher.

ISO/IEC 25010:2011 — official catalogue entry

ISO/IEC 25010:2023 — official catalogue entry